

UNIVERSITÀ DEGLI STUDI DI CATANIA
Anno Accademico 2024 - 2025
Corso di Laurea in Informatica
Prova scritta di Programmazione 1 e laboratorio (A-E / O-Z)
29/01/25

COGNOME e NOME: (IN STAMPATELLO)	
N. MATRICOLA:	

**Non sono consentiti formulari, appunti, libri e calcolatori; non è consentito comunicare con i colleghi; ogni mezzo di comunicazione elettronico deve essere tenuto spento.
Il test va sempre consegnato prima dell'uscita dall'aula.**

Per ciascuna delle seguenti venti domande indicare l'unica risposta corretta.
La prova si considera superata con almeno undici risposte corrette.

1) L'algoritmo di ordinamento Insertion Sort:

- ☐ si basa sull'inserimento di ogni elemento dell'array in un nuovo array ordinato;
- ☐ si basa sull'inserimento del minimo/massimo valore dell'array in un sotto-array ordinato;
- ☐ non è un algoritmo in-place in quanto fa uso di una variabile temporanea;
- ☐ nessuna delle risposte precedenti è corretta.

2) Completare il seguente frame di codice con l'unica istruzione corretta:

```
union data{
    int i;
    double x;
}d;
d.x = 12345678.123456;
d.i = 123456789;
???
```

- ☐ printf("%d %f", d.i, d.x);
- ☐ printf("%f", d.x);
- ☐ printf("%f %d", d.x, d.i);
- ☐ printf("%f %f", (double) d.i, d.x);
- ☐ printf("%d", d.i);

3) Il tipo `long long` del linguaggio C:

- ☐ consente, in qualunque piattaforma, di ottenere un range di valori più ampio del tipo `long`;
- ☐ consente, in qualunque piattaforma, di ottenere un range di valori almeno pari a quello del tipo `long`;
- ☐ se non supportato dalla piattaforma, il codice che contiene il tipo `long long` produrrà errore di compilazione;
- ☐ nessuna delle precedenti risposte è corretta.

4) Il seguente frame di codice:

```
#define B ... // B e' un qualunque intero unsigned maggiore di 1
unsigned x = -1;
while(++x<=B)
    printf("%u", x);
```

- ☐ produrrebbe il seguente output: $0, \dots, B$
 - ☐ produrrebbe B iterazioni
 - ☐ produrrebbe il seguente output: $0, \dots, B + 1$
 - ☐ contiene un errore;
-

5) Con riferimento al seguente frame di codice:

```
unsigned * foo(){
    unsigned *A = malloc(sizeof(unsigned)*10);
    // ...
    return *A;
}
```

- ☐ essendo il puntatore A una variabile locale, l'array sarà deallocato automaticamente con l'uscita dalla funzione stessa;
 - ☐ il puntatore restituito dalla funzione occuperà 4 byte e non 8 byte, come invece accade per i puntatori `double`;
 - ☐ il codice della funzione contiene un errore;
 - ☐ nessuna delle risposte precedenti è corretta;
-

6) Completare la seguente istruzione in linguaggio C per l'inizializzazione di un array di caratteri

```
int n = 5;
char c[n] = ... ;
```

- ☐ `{'x', 'y', 'z'}`
 - ☐ `{'x', 'y', 'z', 'w', 0}`
 - ☐ `{}`
 - ☐ nessuna delle precedenti risposte è corretta;
-

7) Con riferimento alla seguente funzione C:

```
static void foo(){
    // ...
}
```

- ☐ potrà essere invocata da qualunque altra funzione del programma, purchè la funzione `foo()` sia dichiarata **extern** in tutte le translation unit nelle quali è presente una sua invocazione;
 - ☐ essendo la funzione **static**, i dati dichiarati localmente alla funzione conservano i valori tra una chiamata e l'altra;
 - ☐ il qualificatore **static**, per una funzione, non può essere usato in assenza del qualificatore **inline**
 - ☐ nessuna delle risposte precedenti è corretta.
-

- 8) Sia Z un array definito come nel seguente frame di codice. Definire il prototipo di una ipotetica funzione `elab()` che prenda in input un ipotetico array definito come Z .

```
unsigned n = 10, unsigned m = 20, unsigned l=5;
char *Z[n][m][l];
```

```
void elab( ... );
```

- 9) con riferimento alla struttura dati STACK implementata come lista dinamica:

- ☐ è necessario mantenere un puntatore all'elemento in cima ed uno all'elemento alla base dello stack;
 - ☐ l'inserimento di un elemento richiede una prima visita dello stack dalla cima fino alla base;
 - ☐ la cancellazione di un elemento richiede il puntatore alla base dello stack;
 - ☐ l'inserimento di un nuovo elemento richiede il puntatore alla cima dello stack;
 - ☐ nessuna delle precedenti risposte è corretta;
-

- 10) Scegliere l'istruzione corretta per la definizione e contestuale inizializzazione di una stringa s :

- ☐ `char *s[] = "my_string";`
 - ☐ `char s[] = {'m', 'y', '_', 's', 't', 'r', 'i', 'n', 'g', 0};`
 - ☐ `char s*[] = "my_string";`
 - ☐ `char *s[] = {'m', 'y', '_', 's', 't', 'r', 'i', 'n', 'g', 0};`
-

- 11) Completare il seguente frame di codice con una delle quattro proposte elencate in seguito

```
struct record{
    char str[31];
    //...
};
void fill(struct record *ptr, const char *source){
    strcpy( ??? , source);
}
```

- ☐ `&((*ptr).str)`
 - ☐ `&(*ptr).str`
 - ☐ `(*ptr).str`
 - ☐ `ptr->&str[0]`
-

- 12) Completare il seguente frame di codice con una delle quattro proposte elencate in seguito

```
void bar(unsigned n, unsigned m, long **V){
    for(unsigned j=0; j<m; j++)
        for(unsigned i=0; i<n; i++)
            ??? = rand()%10000;
}
```

- ☐ `*(V[i]+j)`
 - ☐ `*V[i+j]`
 - ☐ `*V[i]+j`
 - ☐ `*(V+i)[j]`
-

- 13) Sia P un programma che richiede 3 parametri a linea di comando (argomenti della funzione `main`). Selezionare l'istruzione che permetta di stampare su standard output il numero 30.

```
[Esecuzione del programma P da parte dell'utente]
$ ./P 10 20 30
```

- ☐ `printf("%d", atoi(argv[2]));`
 - ☐ `printf("%s", argv[2]);`
 - ☐ `printf("%d", atof(argv[3]));`
 - ☐ `printf("%s", argv[3]);`
-

- 14) Completare il seguente frame di codice affinché la struttura definita in esso sia *autoreferenziale*

```
struct elem{
    char data[31];
    ???
};
```

- ☐ `struct elem next;`
 - ☐ `struct elem *ptr;`
 - ☐ `typedef struct elem e;`
 - ☐ `struct elem **next`
-

- 15) La seguente dichiarazione:

```
extern struct record r;
```

- ☐ consente di definire una variabile di tipo `struct record` che sia visibile in tutte le funzioni definite nella translation unit nella quale è inserita la dichiarazione;
 - ☐ consente di definire una variabile globale di tipo `struct record` la cui portata sia estesa a tutte le funzioni definite in qualsiasi translation unit del programma;
 - ☐ indica che una variabile locale di tipo `struct record` è definita all'interno di una funzione del programma; con tale dichiarazione, lo scope di tale variabile diverrà globale;
 - ☐ nessuna delle tre risposte precedenti è corretta.
-

- 16) Sia W il nome di un vettore di `double` precedentemente allocato nel segmento HEAP (allocazione dinamica). Il seguente frame di codice

```
free(Z);
Z = NULL;
free(Z);
```

- ☐ provocherebbe errore del compilatore;
 - ☐ provocherebbe un memory leak;
 - ☐ provocherebbe nessun errore;
 - ☐ provocherebbe un errore a tempo di esecuzione;
-

17) Nella codifica di un numero in complemento a due:

- ☐ il bit più significativo rappresenta il segno, gli altri bit il modulo o valore assoluto del numero;
 - ☐ il bit meno significativo rappresenta il segno, gli altri bit il modulo o valore assoluto del numero;
 - ☐ l'intervallo rappresentabile è uguale a $[-2^N, 2^{N-1}]$, dove N e' il numero di bit a disposizione;
 - ☐ l'intervallo rappresentabile è uguale a $[-2^N - 1, 2^{N-1} - 1]$, dove N e' il numero di bit a disposizione;
 - ☐ nessuna delle risposte precedenti è corretta;
-

18) A seguito della seguente istruzione:

```
double y = FLT_MAX * FLT_MAX;
```

- ☐ si avrebbe errore in fase di esecuzione;
 - ☐ si avrebbe errore in fase di compilazione
 - ☐ si avrebbe warning in fase di compilazione;
 - ☐ nessuna delle precedenti risposta è corretta;
-

19) Il seguente frame di codice

```
// sia K una costante positiva
unsigned i=0;
while( i++<K )
    printf("\n %u", rand()-5000);
```

- ☐ permetterebbe di stampare $K + 1$ numeri pseudo-casuali;
 - ☐ permetterebbe di stampare $K - 1$ numeri pseudo-casuali;
 - ☐ permetterebbe di stampare K numeri pseudo-casuali;
 - ☐ contiene un errore;
-

20) Completare il seguente frame di codice per il calcolo del montante M di un capitale C dopo X anni ad un tasso T

```
#define C 1000.0
#define T 0.1
#define X 5

double M = C;
int i = 1;

while( ??? )
    M+=M*T;
```

- ☐ `i++<=X`
- ☐ `++i<=X`
- ☐ `i++<X`
- ☐ `++i<X`